

Ứng dụng phần mềm giải bài toán Tối ưu hóa

Trường Công nghệ thông tin Phenikaa

Đại học Phenikaa

Ứng dụng phần mềm trong tối ưu hóa

- 1 Phần mềm miễn phí và mã nguồn mở
- 2 Phần mềm miễn phí cho mục đích học thuật
- 3 Phần mềm thương mại
- 4 So sánh giữa các công cụ tối ưu hóa
- 5 Tối ưu hóa với Python
 - CVXPY
 - SciPy
 - PuLP

Giới thiệu chung

- Bài toán tối ưu:

Tìm phần tử $x_0 \in A$ sao cho $f(x_0) \leq f(x)$ với mọi $x \in A$,
trong đó $f : A \rightarrow \mathbb{R}$.

- Trong *tối ưu hóa liên tục*, A là một tập con của không gian Euclid $\mathbb{R}^n$, thường được xác định bởi một tập các ràng buộc, là các phương trình hoặc bất phương trình mà các phần tử của A phải thỏa mãn.
- Trong *tối ưu hóa tổ hợp*, A là một tập con của một không gian rời rạc, như chuỗi nhị phân, hoán vị, hoặc tập các số nguyên.

Giới thiệu chung

- Việc sử dụng phần mềm tối ưu hóa đòi hỏi hàm f được định nghĩa trong một ngôn ngữ lập trình phù hợp và được kết nối với phần mềm tối ưu hóa trong quá trình biên dịch hoặc thực thi. Phần mềm tối ưu sẽ cung cấp các giá trị đầu vào thuộc A , mô-đun phần mềm nhận dạng hàm f sẽ trả lại giá trị tính toán $f(x)$ và, trong một số trường hợp, cung cấp thêm thông tin về hàm số như đạo hàm.
- Theo cách này, ta đạt được sự phân tách rõ ràng về trách nhiệm: các mô-đun phần mềm tối ưu khác nhau có thể dễ dàng được thử nghiệm trên cùng một hàm f , hoặc một phần mềm tối ưu cụ thể có thể được sử dụng cho nhiều hàm f khác nhau.

Nguồn tham khảo:

https://en.wikipedia.org/wiki/List_of_optimization_software

1. Phần mềm miễn phí và mã nguồn mở

1.1. Ứng dụng

Tên	Giấy phép	Mô tả
SDMB	BSD	Khung tối ưu hóa phi tuyến sử dụng vi phân tự động.
ASCEND	GPL	Hệ thống mô hình hóa toán học cho các quá trình hóa học.
CUTEr	GPL	Môi trường thử nghiệm cho tối ưu hóa và các trình giải đại số tuyến tính.
GNU Octave	GPL	Gói phần mềm sử dụng ngôn ngữ lập trình cấp cao, chủ yếu dành cho tính toán số; tương thích phần lớn với MATLAB.
Scilab	CeCILL	Gói tính toán số đa nền tảng và ngôn ngữ lập trình cấp cao với khung tối ưu hóa số.

1. Phần mềm miễn phí và mã nguồn mở

1.2. Thư viện phần mềm

Tên	Giấy phép	Mô tả
ALGLIB	GPL	Thư viện tối ưu hóa cấp phép kép (GPL/thương mại) cho các bài toán LP, QP và phi tuyến. Hỗ trợ C++, C#.
COIN-OR	EPL 1.0	Quy hoạch nguyên, quy hoạch tuyến tính và phi tuyến.
Dlib	BSL-1.0	Thư viện tối ưu hóa phi tuyến không ràng buộc hoặc có ràng buộc hộp, viết bằng C++.
GEKKO	MIT	Học máy và tối ưu hóa phương trình vi phân đại số và nguyên hỗn hợp trong Python.
GLPK	GPL	Bộ công cụ quy hoạch tuyến tính GNU với API C.
HiGHS	MIT	Quy hoạch tuyến tính (LP), nguyên hỗn hợp (MIP), và bậc hai lồi (QP).
IPOPT	EPL	Bộ tối ưu hóa phi tuyến quy mô lớn, viết bằng C++ (trước đây là Fortran và C). Một phần của COIN-OR.
MINUIT2	LGPL	Bộ tối ưu hóa không ràng buộc, phát triển tại CERN.
OpenMDAO	Apache	Khung MDAO viết bằng Python, phát triển bởi NASA Glenn với hỗ trợ từ NASA Langley.
SCIP	Apache	Bộ giải MIP và MINLP.
SciPy	BSD	Gói tính toán số Python, hỗ trợ tối ưu hóa.

2. Phần mềm miễn phí cho mục đích học thuật

- AIMMS – Hệ thống mô phỏng để xây dựng ứng dụng hỗ trợ quyết định dựa trên tối ưu hóa với giao diện thân thiện.
- **AMPL** – Ngôn ngữ mô hình hóa đại số mạnh mẽ để xây dựng và giải các bài toán tối ưu hóa quy mô lớn.
- APMonitor – Miễn phí cho mục đích học thuật và thương mại, tích hợp với Python và MATLAB, tối ưu hóa động và điều khiển, chuyên về mô hình hóa quy trình phi tuyến.
- ASTOS – Phần mềm tối ưu hóa quỹ đạo và phân tích nhiệm vụ trong các ứng dụng hàng không vũ trụ.
- **CPLEX** – Bộ giải (solver) của IBM cho các bài toán quy hoạch tuyến tính, nguyên hỗn hợp và quy hoạch bậc hai hiệu suất cao.
- Couenne – Trình giải mã nguồn mở cho các bài toán quy hoạch phi tuyến nguyên hỗn hợp không lồi, theo giấy phép Eclipse Public License.
- **CVX** – Giao diện mô hình hóa trong MATLAB dành cho tối ưu hóa lồi, là công cụ trung gian để xây dựng mô hình.
- FICO Xpress – Bộ công cụ tối ưu hóa cho bài toán tuyến tính, phi tuyến và nguyên hỗn hợp với hiệu suất cao.
- Galahad library – Thư viện dựa trên Fortran để giải các bài toán tối ưu hóa phi tuyến.
- Gekko – Gói Python để tối ưu hóa động và điều khiển dự đoán dựa trên mô hình.

2. Phần mềm miễn phí cho mục đích học thuật

- **Gurobi Optimizer** – Miễn phí cho người dùng học thuật, bộ giải hiệu suất cao cho các bài toán quy hoạch tuyến tính, bậc hai và nguyên hỗn hợp.
- **LIONsolver** – Phần mềm tích hợp cho khai phá dữ liệu, trí tuệ kinh doanh, phân tích và mô hình hóa, sử dụng tối ưu hóa thông minh và học máy.
- **MIDACO** – Gói phần mềm tối ưu số, dựa trên tính toán tiến hóa.
- **MINTO** – Trình giải quy hoạch nguyên sử dụng thuật toán phân nhánh và giới hạn; miễn phí cho người dùng cá nhân.
- **MOSEK** – Phần mềm tối ưu quy mô lớn; giải các bài toán tuyến tính, bậc hai, nón và phi tuyến lồi, liên tục và số nguyên.
- **OptimJ** – Ngôn ngữ mô hình hóa dựa trên Java để xây dựng và giải các bài toán tối ưu hóa; phiên bản miễn phí hỗ trợ lp_solve (tuyến tính), GLPK và định dạng LP hoặc MPS.
- **PottersWheel** – Ước lượng tham số trong phương trình vi phân thường để ước lượng tham số và tối ưu hóa trong mô hình sinh học hệ thống (bộ công cụ MATLAB miễn phí cho học thuật).
- **Pyomo** – Bộ công cụ Python để xây dựng mô hình tối ưu hóa và giải các bài toán tối ưu hóa phức tạp.
- **UFO** – Gói Fortran cho tối ưu hóa số, mô phỏng và giải các bài toán quy hoạch toán học với tính linh hoạt cao.
- **WORHP** – Bộ giải cho các bài toán tối ưu hóa phi tuyến quy mô lớn với các thuật toán số bền vững.

3. Phần mềm thương mại

- AIMMS – Hệ thống mô hình hóa tối ưu hóa, bao gồm giao diện GUI.
- ALGLIB – Thư viện tối ưu hóa bậc hai và phi tuyến bị ràng buộc, cấp phép GPL/thương mại; hỗ trợ C++ và C#.
- Altair HyperStudy – Thiết kế thí nghiệm và tối ưu hóa thiết kế đa ngành.
- **AMPL** – Ngôn ngữ mô hình hóa cho tối ưu hóa tuyến tính lớn, số nguyên hỗn hợp và phi tuyến.
- ANTIGONE – Trình giải MINLP tối ưu hóa toàn cục xác định.
- APMonitor – Bộ công cụ mô hình hóa và tối ưu hóa cho các bài toán lớn, phi tuyến, vi phân và đại số; hỗ trợ MATLAB, Python, Julia.
- Artelys Knitro – Tối ưu hóa phi tuyến quy mô lớn cho bài toán liên tục và số nguyên hỗn hợp.
- **ASTOS** – Phần mềm tối ưu hóa quỹ đạo và phân tích nhiệm vụ trong các ứng dụng hàng không vũ trụ.
- BARON – Tối ưu hóa các bài toán đại số và phi tuyến số nguyên hỗn hợp.
- COMSOL Multiphysics – Phần mềm mô phỏng vật lý đa trường, đa nền tảng, kết hợp phân tích phần tử hữu hạn.
- **CPLEX** – Trình giải cho bài toán quy hoạch tuyến tính và bậc hai với biến liên tục hoặc số nguyên (MIP).

3. Phần mềm thương mại

- **CVX** – Giao diện mô hình hóa trong MATLAB dành cho tối ưu hóa lồi, là công cụ trung gian để xây dựng mô hình.
- FEATool Multiphysics – Hộp công cụ giao diện người dùng (GUI) cho phân tích phần tử hữu hạn (FEA) trên MATLAB.
- FICO Xpress – Bộ giải cho quy hoạch tuyến tính và bậc hai với biến liên tục hoặc nguyên (MIP).
- FortMP – Quy hoạch tuyến tính và bậc hai.
- FortSP – Quy hoạch ngẫu nhiên (stochastic programming).
- GAMS – Hệ thống mô hình đại số tổng quát (General Algebraic Modeling System).
- **Gurobi Optimizer** – Bộ giải cho quy hoạch tuyến tính và bậc hai với biến liên tục hoặc nguyên hỗn hợp (MIP).
- HEEDS MDO – Tối ưu hóa thiết kế đa ngành sử dụng SHERPA, một thuật toán tối ưu hóa thích nghi, lai.
- IMSL Numerical Libraries – Các thuật toán tối ưu hóa tuyến tính, bậc hai, phi tuyến, và QP/LP thưa, được triển khai trong các ngôn ngữ lập trình chuẩn như C, Java, C# .NET, Fortran, và Python.
- IOSO – (Tối ưu hóa gián tiếp dựa trên tự tổ chức) công nghệ tối ưu hóa phi tuyến đa chiều, đa mục tiêu.
- Kimeme – Nền tảng mở cho tối ưu hóa đa mục tiêu và tối ưu hóa thiết kế đa ngành.

3. Phần mềm thương mại

- **LINDO** – (Bộ tối ưu hóa tuyến tính, tương tác và rời rạc) một gói phần mềm cho quy hoạch tuyến tính, quy hoạch nguyên, quy hoạch phi tuyến, quy hoạch ngẫu nhiên, và tối ưu hóa toàn cục. Tiện ích bổ sung "What's Best!" cho Excel thực hiện tối ưu hóa tuyến tính, nguyên và phi tuyến bằng LINDO.
- **LIONsolver** – Phần mềm tích hợp cho khai thác dữ liệu, phân tích, mô hình hóa, học máy và tối ưu hóa thông minh, cùng với cách tiếp cận trí tuệ kinh doanh phản ứng.
- **modeFRONTIER** – Nền tảng tích hợp cho tối ưu hóa đa mục tiêu và đa ngành, cung cấp kết nối liền mạch với các công cụ kỹ thuật của bên thứ ba, tự động hóa quy trình mô phỏng thiết kế, và hỗ trợ ra quyết định phân tích.
- **Maple** – Tối ưu hóa tuyến tính, bậc hai, phi tuyến, liên tục và nguyên. Tối ưu hóa có ràng buộc và không ràng buộc. Tối ưu hóa toàn cục với hộp công cụ bổ sung.
- **MATLAB** – Giải quyết các bài toán tuyến tính, nguyên, bậc hai, và phi tuyến với Hộp công cụ Tối ưu hóa (Optimization Toolbox); xử lý các bài toán có nhiều cực đại, cực tiểu, và các bài toán tối ưu hóa không trơn; ước lượng và tối ưu hóa tham số mô hình.
- **MIDACO** – Một công cụ phần mềm nhẹ cho tối ưu hóa đơn mục tiêu và đa mục tiêu dựa trên tính toán tiến hóa. Được viết bằng C/C++ và Fortran, có cổng kết nối với Excel, VBA, Java, Python, Matlab, Octave, R, C#, và Julia.
- **Mathematica** – Tối ưu hóa đa biến quy mô lớn, có hoặc không có ràng buộc, tuyến tính, bậc hai, phi tuyến, liên tục, và nguyên.

3. Phần mềm thương mại

- ModelCenter – Môi trường đồ họa để tích hợp, tự động hóa và tối ưu hóa thiết kế.
- MOSEK – Tối ưu hóa tuyến tính, bậc hai, hình nón, và phi tuyến lồi, liên tục và nguyên.
- NAG – Tối ưu hóa tuyến tính, bậc hai, phi tuyến, tổng bình phương của các hàm tuyến tính hoặc phi tuyến; tuyến tính, tuyến tính thưa, phi tuyến, có hoặc không có ràng buộc; tối ưu hóa cục bộ và toàn cục; các bài toán liên tục hoặc nguyên.
- NMath – Quy hoạch tuyến tính, bậc hai và phi tuyến.
- **Octeract Engine** – Bộ giải tối ưu hóa toàn cục xác định (deterministic) cho bài toán MINLP (Mixed-Integer Nonlinear Programming). Có kế hoạch bổ sung các tính năng mới.
- OptimJ – Ngôn ngữ mô hình hóa dựa trên Java. Phiên bản Premium hỗ trợ các bộ giải như Mosek và CPLEX.
- Optimus platform – Nền tảng tích hợp quy trình và tối ưu hóa thiết kế, được phát triển bởi Noesis Solutions.
- **OptiY** – Môi trường thiết kế cung cấp các chiến lược tối ưu hóa hiện đại và các thuật toán xác suất tiên tiến cho phân tích bất định, độ tin cậy, độ bền, phân tích độ nhạy, khai thác dữ liệu và mô hình hóa meta.
- OptiStruct – Công nghệ CAE từng đoạt giải thưởng cho tổng hợp thiết kế ý tưởng và tối ưu hóa cấu trúc.

3. Phần mềm thương mại

- OptQuest – Plugin tối ưu hóa dựa trên metaheuristics cho tối ưu hóa dựa trên mô phỏng, kết hợp với phần mềm mô phỏng sự kiện rời rạc.
- **PottersWheel** – Ước lượng tham số trong các phương trình vi phân thường (hộp công cụ MATLAB, miễn phí cho mục đích học thuật).
- pSeven – Nền tảng phần mềm để tự động hóa mô phỏng và phân tích kỹ thuật, tối ưu hóa đa ngành và khai thác dữ liệu, được phát triển bởi DATADVANCE.
- SAS – Bộ phần mềm do SAS Institute phát triển cho phân tích nâng cao (thống kê, dự báo, học máy, tối ưu hóa, v.v.), trí tuệ kinh doanh, trí tuệ khách hàng, quản lý dữ liệu, quản lý rủi ro và nhiều lĩnh vực khác.
- SmartDO – Tối ưu hóa thiết kế toàn cục đa ngành, chuyên về kỹ thuật có sự hỗ trợ của máy tính (CAE), sử dụng các phương pháp tìm kiếm toàn cục trực tiếp.
- SNOPT – Bộ giải cho các bài toán tối ưu hóa quy mô lớn.
- The Unscrambler – Phần mềm tối ưu hóa công thức sản phẩm và quy trình.
- **TOMLAB** – Hỗ trợ tối ưu hóa toàn cục, quy hoạch nguyên, tất cả các dạng bình phương tối thiểu, quy hoạch tuyến tính, bậc hai và không ràng buộc cho MATLAB. TOMLAB hỗ trợ các bộ giải như CPLEX, SNOPT, KNITRO và MIDACO.
- VisSim – Ngôn ngữ sơ đồ khối trực quan cho mô phỏng và tối ưu hóa các hệ thống động.
- WORHP – Bộ giải thừa quy mô lớn cho tối ưu hóa phi tuyến liên tục.

4. So sánh giữa các công cụ tối ưu hóa

Công cụ	Loại bài toán	Ngôn ngữ	Giấy phép	Tính năng
SciPy	Tuyến tính, phi tuyến	Python	Miễn phí	Tích hợp Python, khoa học dữ liệu
CVX	Tuyến tính, lồi, bậc hai, SDP	MATLAB	Thương mại	Mô hình hóa tối ưu lồi, tích hợp MATLAB
CPLEX	Tuyến tính, bậc hai, nguyên hỗn hợp (MIP)	C, Java, Python	Thương mại	Hiệu suất cao, MIP lớn
AMPL	Tuyến tính, phi tuyến, ngẫu nhiên	Ngôn ngữ riêng	Thương mại	Mô hình hóa mạnh, đa bộ giải
MATLAB	Tuyến tính, phi tuyến, nguyên	MATLAB	Thương mại	Hộp công cụ toàn diện
Gurobi	Tuyến tính, bậc hai, MIP	C, Python, MATLAB	Thương mại	Tính toán song song
R	Tuyến tính, phi tuyến, thống kê	R	Miễn phí	Phân tích thống kê, tối ưu hóa
Maple	Tuyến tính, phi tuyến, nguyên	Maple	Thương mại	Tối ưu toàn cục, tính toán ký hiệu
LINDO	Tuyến tính, nguyên, phi tuyến	LINDO, Excel	Thương mại	Add-in Excel, dễ sử dụng

5. Tối ưu hóa với Python

Xét bài toán:

$$\min_x f(x)$$

Với các ràng buộc:

$$h_i(x) = 0 \quad (\text{Ràng buộc đẳng thức})$$

$$g_j(x) \leq 0 \quad (\text{Ràng buộc bất đẳng thức})$$

$$x_k^L \leq x_k \leq x_k^U \quad (\text{Ràng buộc biên})$$

Các thư viện Python phổ biến

Thư viện cơ bản:

- SciPy (minimize, linprog)
- PuLP (Quy hoạch tuyến tính)

Thư viện nâng cao:

- CVXPY (Tối ưu lồi)
- Pyomo (Mô hình hóa phức tạp)
- GEKKO (Tối ưu động)

So sánh các thư viện

Thư viện	Phù hợp nhất cho
CVXPY	Bài toán lồi
Pyomo	Mô hình phức tạp
SciPy	Mục đích chung
PuLP	LP/MILP đơn giản

- Thư viện Python mã nguồn mở cho **tối ưu hóa lồi**

- Thư viện Python mã nguồn mở cho **tối ưu hóa lồi**
- Đặc điểm nổi bật:
 - Cú pháp trực quan, gần với toán học
 - Hỗ trợ nhiều loại bài toán (LP, QP, SDP,...)
 - Tự động kiểm tra tính lồi (DCP rules)

Giới thiệu CVXPY

- Thư viện Python mã nguồn mở cho **tối ưu hóa lồi**
- Đặc điểm nổi bật:
 - Cú pháp trực quan, gần với toán học
 - Hỗ trợ nhiều loại bài toán (LP, QP, SDP,...)
 - Tự động kiểm tra tính lồi (DCP rules)
- Ứng dụng:
 - Machine Learning
 - Tài chính định lượng
 - Điều khiển tối ưu

Các loại bài toán hỗ trợ

Loại bài toán	Cú pháp CVXPY
Linear Program (LP)	<code>cp.Minimize(c.T@x)</code>
Quadratic Program (QP)	<code>cp.quad_form(x, P)</code>
Semidefinite Program (SDP)	<code>cp.variable((n,n), PSD=True)</code>
Sparse Optimization	<code>cp.sum_entries(cp.abs(x))</code>

Lưu ý

CVXPY tự động kiểm tra tính lồi thông qua **DCP (Disciplined Convex Programming)**

Cấu trúc một bài toán CVXPY

```
1      import cvxpy as cp
2
3      # Variable declaration
4      x = cp.Variable(n)
5
6      # Constraints
7      constraints = [f(x) <= 0, ...]
8
9      # Objective
10     objective = cp.Minimize(cp.norm(A@x - b))
11
12     # Solve the problem
13     prob = cp.Problem(objective, constraints)
14     prob.solve()
```

Ví dụ: Bài toán bình phương tối thiểu

Phát biểu bài toán:

Tối thiểu hóa $\|Ax - b\|_2^2$ (tổng bình phương sai số)

Tìm nghiệm tối ưu tương ứng x^* , trong đó $r = Ax^* - b$ được gọi là phần dư (residual).

Ví dụ: Bài toán bình phương tối thiểu

```
1      import cvxpy as cp
2      import numpy as np
3
4      # Data
5      m, n = 30, 20
6      np.random.seed(1)
7      A = np.random.randn(m, n)
8      b = np.random.randn(m)
9      # Variable and Problem
10     x = cp.Variable(n)
11     objective = cp.Minimize(cp.sum_squares(A @ x
12                                     - b))
13     constraints = [0 <= x, x <= 1]
14     prob = cp.Problem(objective, constraints)
15     # Solve the problem
16     result = prob.solve()
17     print(result) # Output: 19.83126370644502
```

Ví dụ: Bài toán tối ưu hóa danh mục đầu tư

Bài toán

Tối đa hóa lợi nhuận điều chỉnh rủi ro:

$$\text{maximize } \mu^T w - \gamma w^T \Sigma w$$

với ràng buộc:

$$1^T w = 1, \quad w \geq 0$$

- μ : vector lợi nhuận kỳ vọng
- Σ : ma trận hiệp phương sai
- γ : tham số ưu tiên rủi ro

Ví dụ: Bài toán tối ưu hóa danh mục đầu tư

```
1      import numpy as np
2      import scipy.sparse as sp
3      import cvxpy as cp
4
5      # Generate random data
6      np.random.seed(1); n = 10
7      mu = np.abs(np.random.randn(n, 1))
8      Sigma = np.random.randn(n, n)
9      Sigma = Sigma.T.dot(Sigma)
```

Ví dụ: Bài toán tối ưu hóa danh mục đầu tư

```
1      # Solve the problem using CVXPY
2      w = cp.Variable(n)
3      gamma = cp.Parameter(nonneg=True)
4      ret = mu.T @ w
5      risk = cp.quad_form(w, Sigma)
6      objective = cp.Maximize(ret - gamma * risk)
7      constraints = [cp.sum(w) == 1, w >= 0]
8      prob = cp.Problem(objective, constraints)
9      gamma.value = 0.2
10     prob.solve()
11     # Output: 1.5000976202809573
```

Ví dụ: Bài toán tối ưu lồi

```
1      import cvxpy as cp
2
3      # Define variables
4      x = cp.Variable()
5      y = cp.Variable()
6
7      # Define constraints
8      constraints = [x + y == 1, x - y >= 2]
9
10     # Define the objective function
11     objective = cp.Minimize(x**2 + y**2)
12
13     # Formulate the problem
14     prob = cp.Problem(objective, constraints)
```

Ví dụ: Bài toán tối ưu lồi

```
1      # Solve the problem
2      result = prob.solve()
3      print(f"Optimal value: {result}")
4      print(f"x: {x.value}, y: {y.value}")
5
6      # >>> Optimal value: 2.5
7      # x: 1.5, y: -0.5000000000000001
```

Ưu điểm của CVXPY

- **Dễ sử dụng**

- Cú pháp gần với toán học
- Không cần chuyển đổi sang dạng chuẩn

- **Linh hoạt**

- Hỗ trợ nhiều loại bài toán
- Dễ dàng kết hợp với NumPy

- **Mạnh mẽ**

- Giải được bài toán quy mô lớn
- Nhiều solver backend (ECOS, SCS,...)

Các ứng dụng khác

- **Xử lý ảnh:**

- Khôi phục ảnh (inpainting)
- Giảm nhiễu

Original Image



Corrupted Image



In-Painted Image



Difference Image



- **Điều khiển tối ưu:**

- Lập quỹ đạo tên lửa
- Điều khiển robot



Giới thiệu SciPy Optimization

- Thư viện mạnh mẽ trong hệ sinh thái SciPy
- Hỗ trợ nhiều loại bài toán tối ưu:
 - Tối ưu không ràng buộc/ràng buộc
 - Tìm nghiệm (root-finding)
 - Curve fitting
- Ưu điểm:
 - Dễ sử dụng
 - Tích hợp nhiều thuật toán (BFGS, Nelder-Mead, SLSQP...)

Các hàm chính trong scipy.optimize

Các hàm tiêu biểu

- `minimize()`: Tối thiểu hóa hàm mục tiêu
- `least_squares()`: Bình phương tối thiểu
- `linprog()`: Tối ưu tuyến tính
- `minimize-scalar()`: Tối ưu hàm một biến

minimize: Tối ưu không ràng buộc

Ví dụ cú pháp minimize

```
from scipy.optimize import  
    minimize  
result = minimize(fun, x0, method  
                  = 'BFGS')
```

minimize: Tối ưu không ràng buộc

Bài toán

Tìm cực tiểu hàm $f(x) = x_0^2 + x_1^2$

```
def objective(x):  
    return x[0]**2 + x[1]**2  
  
result = minimize(objective, [1,  
    1], method='BFGS')  
print(result.x) # Output: [~0.0,  
    ~0.0]
```

- $x_0=[1, 1]$: Điểm khởi tạo
- $\text{method}='BFGS'$: Thuật toán gradient-based
- Kết quả gần đúng $[0.0, 0.0]$

minimize với ràng buộc

- Ràng buộc tuyến tính hoặc phi tuyến
- Phương pháp: SLSQP, trust-constr

Ràng buộc đoạn

Ví dụ với ràng buộc $0 \leq x_0 \leq 1; -0.5 \leq x_1 \leq 2.0$

```
from scipy.optimize import Bounds
bounds = Bounds([0, -0.5], [1.0,
                           2.0])
```

minimize với ràng buộc

Ràng buộc tuyến tính: Các ràng buộc $x_0 + 2x_1 \leq 1$ và $2x_0 + x_1 = 1$ có thể được viết lại dưới dạng ràng buộc tuyến tính chuẩn như sau:

$$\begin{bmatrix} -\infty \\ 1 \end{bmatrix} \leq \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} \leq \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

```
from scipy.optimize import  
    LinearConstraint  
linear_constraint =  
    LinearConstraint([[1, 2], [2,  
        1]], [-np.inf, 1], [1, 1])
```

minimize với ràng buộc

Ràng buộc phi tuyến

$$c(x) = \begin{bmatrix} x_0^2 + x_1 \\ x_0^2 - x_1 \end{bmatrix} \leq \begin{bmatrix} 1 \\ 1 \end{bmatrix},$$

với ma trận Jacobian:

$$J(x) = \begin{bmatrix} 2x_0 & 1 \\ 2x_0 & -1 \end{bmatrix},$$

và sự kết hợp tuyến tính của Hessians:

$$H(x, v) = \sum_{i=0}^1 v_i \nabla^2 c_i(x) = v_0 \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix} + v_1 \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix},$$

được sử dụng bởi `NonlinearConstraint` object.

minimize với ràng buộc

```
def cons_f(x):  
    return [x[0]**2 + x[1], x[0]**2 -  
            x[1]]  
def cons_J(x):  
    return [[2*x[0], 1], [2*x[0],  
                           -1]]  
def cons_H(x, v):  
    return v[0]*np.array([[2, 0], [0,  
                              0]]) + v[1]*np.array([[2,  
                              0], [0, 0]])  
from scipy.optimize import  
    NonlinearConstraint  
nonlinear_constraint =  
    NonlinearConstraint(cons_f, -  
                        np.inf, 1, jac=cons_J, hess=  
                        cons_H)
```

minimize với ràng buộc

Giải bài toán

```
x0 = np.array([0.5, 0])
res = minimize(rosen, x0, method=
    'trust-constr', jac=rosen_der
    , hess=rosen_hess,
    constraints=[linear_constraint,
        nonlinear_constraint],
    options={'verbose': 1}, bounds=
        bounds)
print(res.x)
# [0.41494531 0.17010937]
```

Tối ưu hóa hàm một biến với `minimize_scalar`

- Tối ưu hàm $f : \mathbb{R} \rightarrow \mathbb{R}$

Ví dụ: Tìm giá trị nhỏ nhất của hàm $J_1(x)$ gần $x = 5$, giới hạn trong đoạn $[4, 7]$. Ta dùng:

```
from scipy.special import j1
res = minimize_scalar(j1, bounds
                      =(4, 7), method='bounded')
res.x
# Output: 5.33144184241
```

Kết luận: Nghiệm tối ưu là $x_{\min} = 5.3314$.

linprog: Bài toán tối ưu tuyến tính

- Giải bài toán

$$\min c^T x \quad \text{s.t. } Ax \leq b$$

```
from scipy.optimize import  
    linprog  
  
c = [-1, -2]  
A = [[2, 1], [1, 1]]  
b = [20, 16]  
res = linprog(c, A_ub=A, b_ub=b,  
              method='highs')  
print(res.x)
```

least_squares cho fitting dữ liệu

Cho dữ liệu mô phỏng theo công thức:

$$y_i = 3x_i + 2 + \varepsilon$$

với $\varepsilon \sim \mathcal{N}(0, 1)$ Yêu cầu: Tìm các hệ số a và b sao cho đường thẳng:

$$y = ax + b$$

phù hợp nhất với tập dữ liệu (theo tiêu chí bình phương sai số).

least_squares cho fitting dữ liệu

```
import numpy as np
from scipy.optimize import
    least_squares

# Data
x_data = np.linspace(0, 10, 20)
y_data = 3.0 * x_data + 2.0 + np.
    random.normal(size=x_data.
        size)

# Model:  $y = ax + b$ 
def residuals(params, x, y):
    a, b = params
    return a * x + b - y
x0 = [1, 1]
res = least_squares(residuals, x0
    , args=(x_data, y_data))
print(res.x)
```

least_squares cho fitting dữ liệu

Hàm cần tối thiểu hóa là:

$$\min_{a,b} \sum_{i=1}^n (ax_i + b - y_i)^2$$

Hàm least_squares sẽ nội suy để tìm ra cặp (a^*, b^*) sao cho sai số bình phương là nhỏ nhất.

PuLP: Quy hoạch nguyên hỗn hợp

PuLP là một thư viện Python mã nguồn mở dùng để mô hình hóa và giải các bài toán quy hoạch tuyến tính (Linear Programming - LP) và quy hoạch nguyên (Integer Programming - IP).

Ứng dụng:

- Tối ưu hóa chuỗi cung ứng (Supply Chain)
- Lập lịch (Scheduling)
- Lập kế hoạch sản xuất (Production Planning)
- Phân phối tài nguyên (Resource Allocation)
- Và nhiều bài toán liên quan tới tối ưu hóa khác

PuLP: Quy hoạch nguyên hỗn hợp

Tối đa hóa $z = 4x + 3y$ với ràng buộc:

$$\begin{cases} 2x + y \leq 20 \\ 4x - 5y \geq -10 \\ x \geq 0, y \geq 3 \end{cases}$$

PuLP: Quy hoạch nguyên hỗn hợp

```
import pulp
# Initialize the model
model = pulp.LpProblem("
    Maximize_Z", pulp.LpMaximize)
# Variables
x = pulp.LpVariable("x", lowBound
    =0)
y = pulp.LpVariable("y", lowBound
    =3)
# Objective
model += 4 * x + 3 * y
# Constraints
model += 2 * x + y <= 20
model += 4 * x - 5 * y >= -10
# Solve the Problem
model.solve()
# Output
print("x =", x.value())
print("y =", y.value())
print("Value of Z =", pulp.value(
    model.objective))
```

Lựa chọn thư viện

- Bài toán đơn giản:
 - SciPy.optimize cho NLP
 - PuLP cho LP
- Bài toán lồi:
 - CVXPY (cú pháp rõ ràng)
 - CVXOPT cho QP
- Mô hình phức tạp:
 - Pyomo (hỗ trợ nhiều loại bài toán)
 - GEKKO cho hệ thống động

Bài tập

Sử dụng Python để tìm nghiệm tối ưu cho các bài toán sau:

Bài tập 1:

Tìm giá trị nhỏ nhất của hàm mục tiêu:

$$f(x) = 2x_1 + 2x_2 + 3x_3$$

với các ràng buộc:

$$\begin{cases} x_1 + x_2 + x_3 \geq 1 \\ 2x_1 - x_2 + 3x_3 \leq 5 \\ x_1 \geq 0, x_2 \geq 0, x_3 \geq 0 \end{cases}$$

Bài tập

Bài tập 2:

Tìm giá trị nhỏ nhất của hàm mục tiêu:

$$f(x) = x_1 + x_2$$

với các ràng buộc:

$$\begin{cases} x_1^2 + x_2^2 \leq 1 \\ x_1 + x_2 \geq 0 \\ x_1 \geq -1, x_2 \geq -1 \end{cases}$$

Bài tập 3:

Tìm giá trị nhỏ nhất của hàm mục tiêu:

$$f(x) = x_1^2 + 2x_2^2 + 3x_3^2 - 2x_1x_2 - 2x_2x_3$$

- Địa chỉ download phần mềm:
 - CVXPY: <https://www.cvxpy.org/>
 - PuLP: <https://coin-or.github.io/pulp/>
 - Scipy: <https://docs.scipy.org/doc/scipy/tutorial/>
- Tài liệu tham khảo:
 - *"Hands-On Mathematical Optimization with Python"* (2025), K. Postek, A. Zocca, J.A.S. Gromicho, J.C. Kantor, Cambridge University Press.
 - *"Convex Optimization"* (2009), S. Boyd, L. Vandenberghe, Cambridge University Press.

BÀI TẬP NHÓM:

Sử dụng công cụ phần mềm tối ưu giải các bài tập trong slide bài giảng các chương đã học:

1. Giới thiệu công cụ mà nhóm sử dụng, ưu nhược điểm.
2. Trình bày lời giải từng bài, thêm nhận xét (nếu có).
3. Mở rộng giải quyết các bài toán khác hoặc bài toán thực tế (nếu có).